

任务 1：AI 原生应用的质量属性场景分析

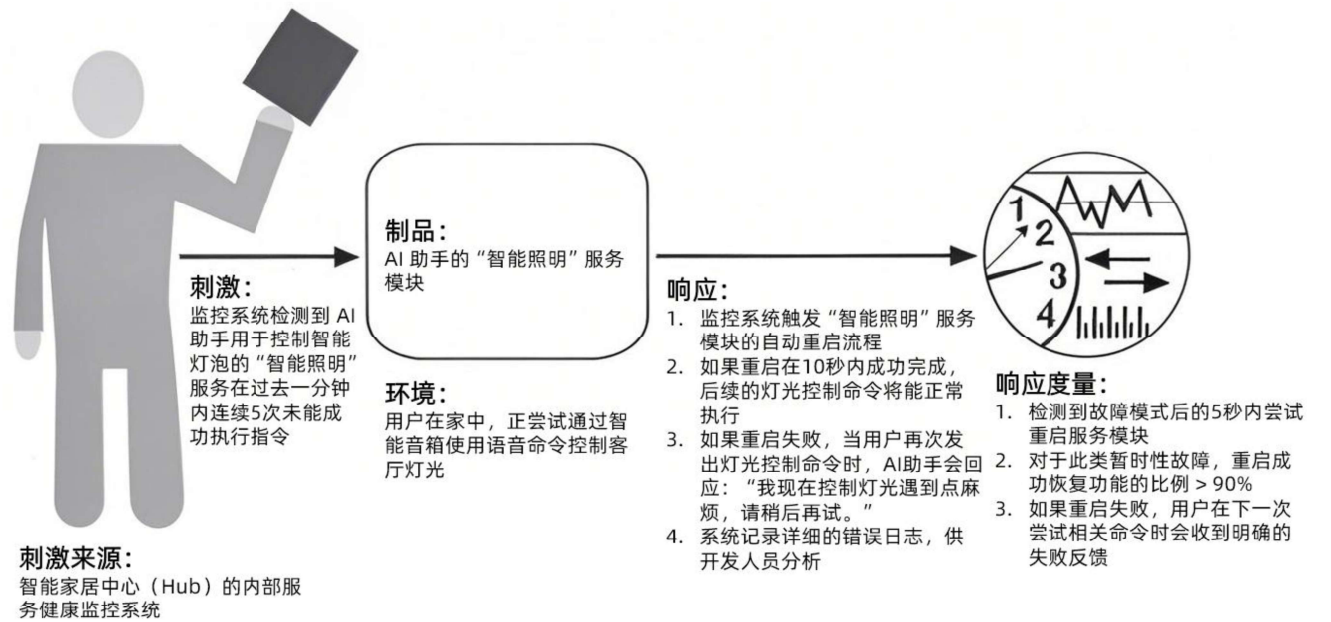
1.1 可用性（Availability）

理由：AI 智能体需要能持续运行，并且随时响应用户请求或环境变化以执行任务。无论是用于客户服务、自动化流程还是个人助理，其服务中断可能导致用户不满、业务损失或错失机会。高可用性确保智能体在其需要时能够可靠地提供服务。

通用场景：

场景元素	可能的值
刺激来源	内部/外部：硬件故障、软件错误、网络中断、依赖服务不可用
刺激	故障：遗漏、崩溃、不正确计时、不正确响应
制品	AI 智能体系统、特定的智能体组件、底层基础设施
环境	正常运行、启动、关闭、修复模式、降级运行、过载运行
响应	防止故障演变成失效；检测故障（记录日志、通知相关实体）；从故障中恢复（禁用故障源、暂时不可用、修复/屏蔽故障、降级运行）
响应度量	可用性百分比、故障检测时间、故障修复时间、系统可处于降级模式的时间/时间间隔、系统阻止或处理的某类故障的比例/速率

具体场景：



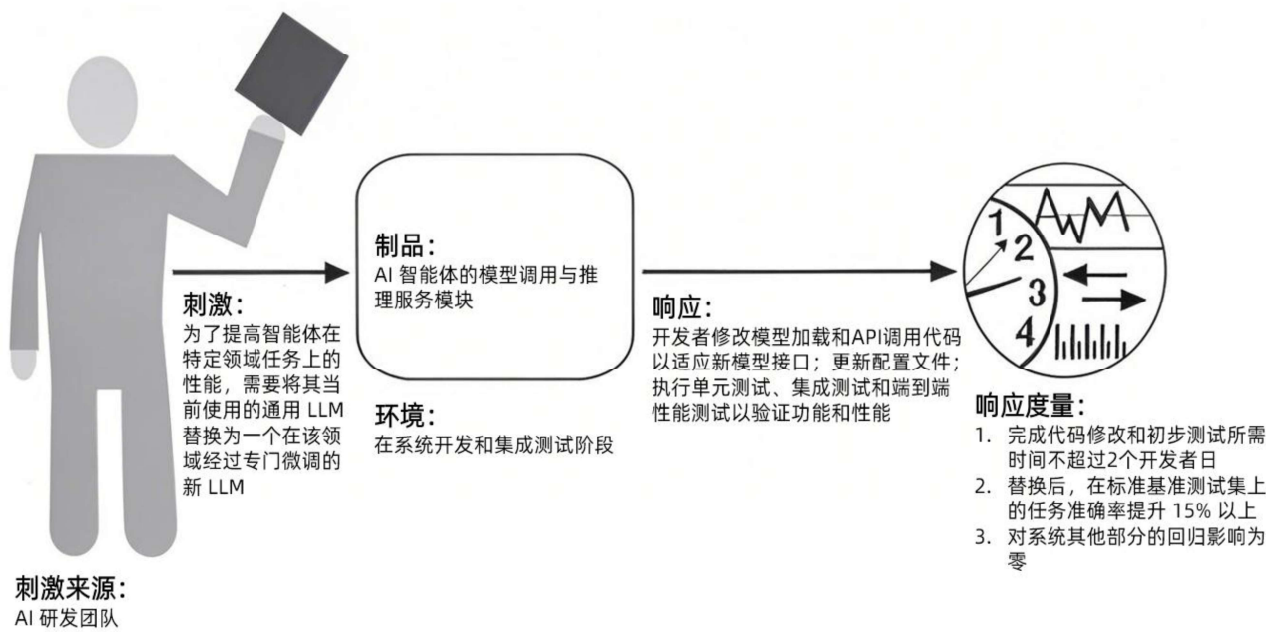
1.2 可修改性（Modifiability）

理由：AI 领域发展迅速，新的算法、模型和技术层出不穷。研发者需要不断更新 AI 智能体的核心模型、算法、知识库或集成新的外部工具，以保持竞争力、适应新需求或修复缺陷。因此 AI 智能体应用架构需要支持这种快速迭代和演进，降低修改的成本和风险。

通用场景：

场景元素	可能的值
刺激来源	开发者、产品经理、系统管理员
刺激	需要添加新功能、修改现有功能、删除功能、适应新的技术标准或平台、修复缺陷
制品	AI 智能体的特定模块、系统架构、用户界面、部署脚本
环境	开发时、编译时、部署时、运行时
响应	定位需要修改的模块；进行修改；测试修改；部署修改
响应度量	修改所需时间/人力成本、受修改影响的模块数量、测试的复杂性/覆盖率、部署的难易度

具体场景：



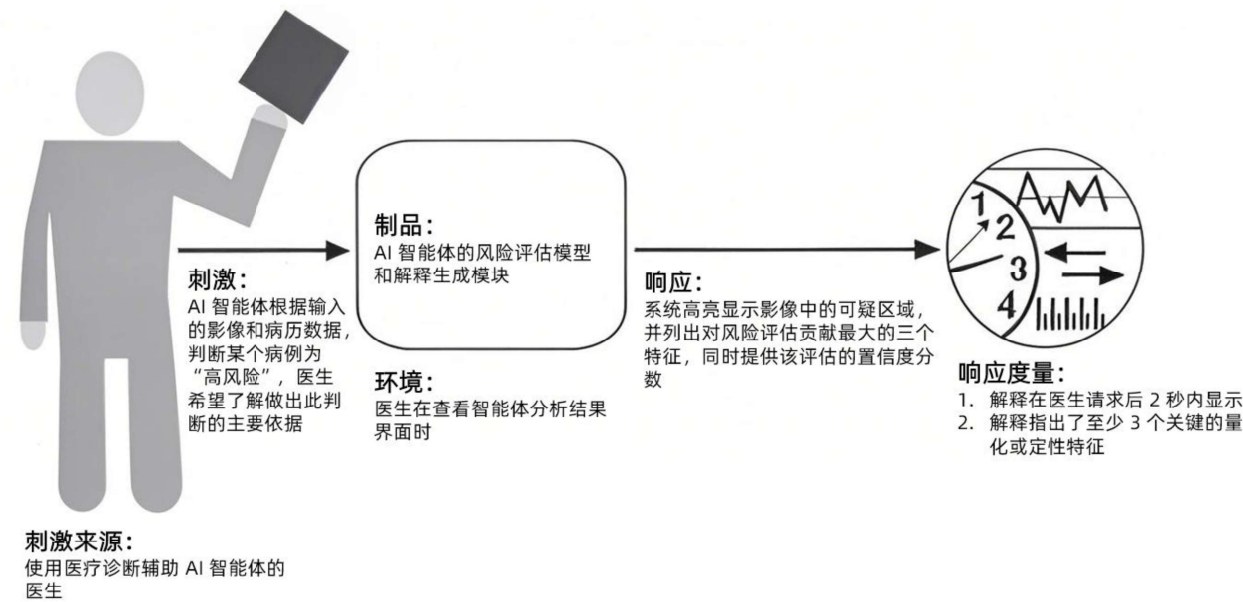
1.3 可解释性（Explainability）

理由：AI 智能体，特别是那些具有高度自主性的智能体，其决策过程往往像一个“黑箱”。为了建立用户信任、满足合规性要求、方便调试和验证系统行为的正确性与公平性，智能体需要能够解释其做出特定决策或推荐的原因。

通用场景：

场景元素	可能的值
刺激来源	最终用户、开发者、审计员、监管机构
刺激	请求对 AI 智能体的某个特定输出（决策、预测、推荐）进行解释或说明理由。
制品	AI 智能体的决策逻辑模块、推理引擎、以及用于生成解释的特定组件或模型。
环境	正常运行期间，用户与智能体交互后，或在进行审计/调试时。
响应	系统提供对其输出的解释，可能形式包括：关键影响因素、决策规则或路径的可视化、置信度分数、相似案例、自然语言描述、反事实解释。
响应度量	解释生成时间、解释的清晰度/可理解性（用户评分）、解释的忠实度（解释与模型实际行为的一致性）、解释覆盖的决策类型比例、满足监管要求的程度。

具体场景：



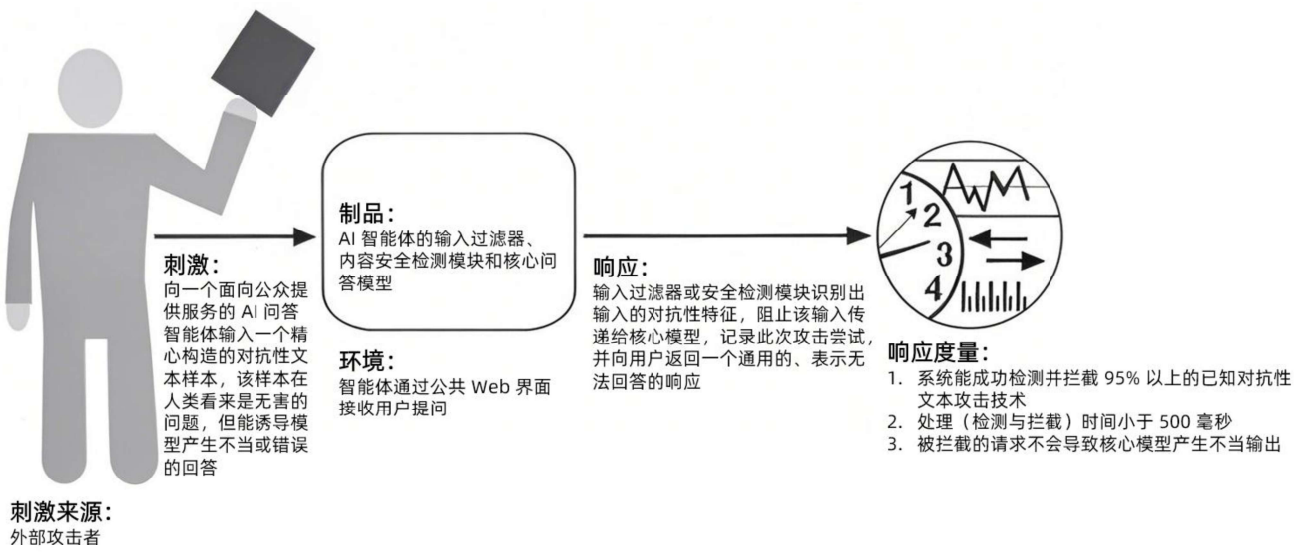
1.4 鲁棒性（Robustness）

理由：AI 智能体需要在现实世界的各种条件下稳定运行，包括噪声数据、意外输入、环境变化甚至恶意攻击。鲁棒性确保智能体在遇到非理想或预期外情况时，仍能保持其性能水平，避免做出错误或有害的决策，并能安全地降级或处理异常。这对于需要与不可预测环境交互或处理潜在对抗性输入的智能体至关重要。

通用场景：

场景元素	可能的值
刺激来源	外部环境、用户、潜在的攻击者
刺激	接收到预期外的数据、格式错误的的数据、噪声干扰的数据、对抗性构造的旨在欺骗模型的输入、系统运行环境发生剧烈变化。
制品	AI 智能体的输入处理模块、数据预处理流程、核心 AI 模型、输出验证机制。
环境	正常运行，但也可能是在压力测试、安全测试或遇到意外操作条件下。
响应	系统能够检测到异常/对抗性输入；拒绝处理恶意输入；在输入扰动下保持性能稳定；从瞬时干扰中恢复；记录安全事件。
响应度量	在特定噪声水平下性能下降幅度、成功抵御的对抗性攻击类型/比例、错误输入的拒绝率、从干扰恢复到正常状态的时间、误报/漏报对抗性输入的比率。

具体场景：



任务 2：质量属性的策略与策略

质量属性	策略	战术	收益	惩罚/对其他属性的影响
可用性	故障检测	心跳	能够快速检测到无响应的组件 实现相对简单	增加网络负载（性能） 心跳间隔设置不当会导致检测延迟或误报 无法检测组件内部逻辑错误
		条件监控	可以检测更广泛的故障类型 提供更丰富的系统状态信息	实现更复杂 可能需要更多系统资源（性能） 监控逻辑本身可能出错
	故障恢复	主动冗余	故障切换几乎是瞬时的，恢复时间极短 所有副本状态一致	资源成本高 实现复杂性高，需要处理并发和投票机制 可能增加写操作的延迟（性能）
		被动冗余	资源成本相对较低 实现相对简单	故障切换有延迟，需要时间同步状态 可能存在数据丢失 主备切换逻辑可能引入复杂性
可修改性	本地化修改	使用抽象接口	降低模块间耦合度 隐藏实现细节，使修改一个模块不易影响其他模块 提高可测试性	可能引入性能开销 接口设计需要仔细考虑，否则可能成为修改的瓶颈 增加设计的复杂性
		封装/信息隐藏	将易变部分隔离，限制修改影响范围 提高模块内聚性	过度封装可能导致接口复杂或不灵活 需要清晰定义模块边界和职责
	推迟绑定时间	使用配置文件/参数化	无需重新编译即可修改系统行为 提高灵活性和可部署性	配置管理变得复杂 运行时读取配置可能引入微小性能开销 配置错误可能导致运行时失败（可用性）
		容器化与编排	易于部署不同版本的服务 简化依赖管理 支持蓝绿部署/金丝雀发布等策略，降低更新风险	引入额外的基础设施复杂性 容器本身有资源开销（性能） 跨容器通信可能增加延迟（性能）

质量属性	策略	战术	收益	惩罚/对其他属性的影响
可解释性	提供局部解释	使用可解释模型	模型本身决策逻辑透明，易于理解和验证 计算开销较低	对于复杂问题，简单模型可能性能不足（功能性/准确性） 可能无法捕捉数据中的复杂非线性关系
		应用事后解释技术	可以为复杂的“黑箱”模型提供解释 与模型类型无关，应用广泛	解释的计算成本可能很高（性能） 解释的保真度可能存疑 解释本身可能难以理解或被误解 需要额外实现和维护解释组件（可修改性）
	提供全局解释	特征重要性分析	帮助理解哪些输入特征对模型决策整体影响最大 有助于模型调试和特征工程	无法解释单个预测的原因 可能忽略特征间的交互作用 不同计算方法可能结果不同
		模型可视化	提供对模型行为的直观理解 有助于发现模型学习到的模式或潜在问题	主要适用于低维数据或特定类型的模型 高维数据可视化困难 可能需要专门的工具和专业知识来解读（易用性）
鲁棒性	输入验证与净化	输入防护栏	阻止已知的不良输入到达核心模型 提高安全性 有助于满足合规要求	可能误拦正常输入（功能性） 需要持续更新防护规则以应对新威胁（可维护性） 可能会引入处理延迟（性能）
		数据增强	提高模型对输入变化的容忍度 提升模型泛化能力	可能增加训练时间和计算成本（性能） 不恰当的增强可能降低模型在干净数据上的性能 需要仔细设计增强策略
	对抗性防御	对抗训练	提高模型对特定类型对抗性攻击的抵抗力 使模型决策边界更平滑	显著增加训练时间和计算成本（性能） 可能降低模型在干净数据上的标准准确性 防御效果通常只针对训练中使用的攻击类型（泛化性有限）
		输出验证/过滤	检查模型输出是否合理、安全或符合预期模式 可以作为最后一道防线，阻止不当内容生成	可能误过滤掉正常的输出（功能性） 设计有效的验证规则可能很困难，且需要领域知识 可能会引入输出延迟（性能）

任务 3：设计决策

3.1 针对可解释性

职责分配（Allocation of Responsibilities）：

- 决策 1：**划分一个专门的解释生成模块，用于整合来自 AI 的原始信息并转化为人类可理解的解释
理由：能集中管理解释逻辑，确保解释的一致性和质量。并且方便独立更新
- 决策 2：**为核心 AI 模块定义解释接口，强制要求其输出决策依据元数据（如特征权重、置信度）
理由：为解释生成模块提供必要的的数据，确保解释的准确性和可靠性

协调模型（Coordination Model）：

- 决策 1：**核心决策模块主动通知解释模块预先准备或缓存相关解释元素
理由：缓解解释生成可能耗时较长的情况，提升用户体验和系统性能
- 决策 2：**采用“请求-响应”机制实现决策模块与解释模块的解耦，解释请求触发时同步返回决策路径信息
理由：松耦合设计支持解释模块独立演进，适应不同用户对解释粒度的需求，同时避免解释生成阻塞核心决策流程

数据模型（Data Model）：

- 决策 1：**设计解释元数据存储结构，包含决策上下文、推理路径、可信度指标等
理由：可以为解释生成提供完整的历史数据，支持事后审计和实时解释查询，同时有利于实现解释内容的版本管理和对比分析
- 决策 2：**定义标准化的 AI 解释模式，统一解释的结构和内容
理由：确保解释的一致性和可理解性，便于不同角色的使用和审计

资源管理（Resources Management）：

- 决策 1：**为解释生成预留专用资源，并设计资源监控动态调整分配策略
理由：避免与模型推理竞争资源，防止因资源不足导致解释延迟或失败
- 决策 2：**对常用解释设计缓存机制，设置缓存失效策略
理由：减少重复计算，降低解释生成时间和负载，提升响应速度

架构元素间的映射（Mapping Among Architectural Elements）：

- 决策 1：**将解释生成模块设计为可独立部署和扩展的服务，通过接口与 AI 模型服务交互
理由：有利于模块化和独立演进
- 决策 2：**将轻量级解释模块嵌入用户设备端，复杂解释逻辑交由服务器端处理
理由：提升响应速度和效率

绑定时间（Binding Time）：

- 决策 1：**将解释策略设计为运行时可配置参数，通过配置文件或 API 调整
理由：支持根据用户角色、使用场景动态切换解释模式，提升了系统的灵活性
- 决策 2：**支持动态加载解释模型/插件
理由：提升系统在解释能力上的可修改性，便于集成新的解释技术

技术选择（Choice of Technology）：

- 决策 1：**选择支持可视化解释的前端框架，将复杂解释逻辑转化为用户可理解的图表
理由：通过可视化提升解释的可理解性和美观度
- 决策 2：**采用可解释人工智能工具链作为解释模块的核心技术
理由：可以降低成本，并能加速开发，降低风险

3.2 鲁棒性

职责分配 (Allocation of Responsibilities) :

- 决策 1:** 在系统入口处实现专门的输入验证与净化层
理由: 集中处理不可信输入, 防止噪声数据污染
- 决策 2:** 设立异常检测组件, 持续监测智能体的运行状态和输出
理由: 能保护核心系统, 提高可靠性

协调模型 (Coordination Model) :

- 决策 1:** 在分布式系统中采用重试-超时-熔断机制
理由: 防止故障扩散, 提高系统在依赖服务出问题时的整体稳定性
- 决策 2:** 设计自适应负载分配算法, 根据组件状态动态调整
理由: 通过动态负载均衡减轻故障节点压力, 利用健康节点的冗余能力维持系统整体性能, 提升应对突发负载或部分组件失效的能力

数据模型 (Data Model) :

- 决策 1:** 引入数据清洗与增强模块, 可以用于噪声过滤、数据归一化、对抗样本注入训练等预处理步骤
理由: 通过数据增强提升模型对噪声的鲁棒性, 减少因输入不一致导致的错误
- 决策 2:** 维护一个动态更新的安全数据库, 记录已知的攻击模式和异常特征
理由: 有助于系统识别和抵御已知威胁

资源管理 (Resources Management) :

- 决策 1:** 预留一部分的资源作为缓冲区, 用于处理突发异常负载
理由: 避免系统因资源耗尽而崩溃, 确保异常场景下的服务连续性
- 决策 2:** 对高频异常输入源进行速率限制
理由: 防止恶意请求耗尽系统资源

架构元素间的映射 (Mapping Among Architectural Elements) :

- 决策 1:** 采用隔离容器部署, 将易受攻击的组件与核心逻辑隔离
理由: 可以限制故障扩散范围, 增强安全性
- 决策 2:** 将输入验证和净化等安全组件部署在系统边界, 作为所有外部交互的必经之路。
理由: 能提高安全性, 形成防御层

绑定时间 (Binding Time) :

- 决策 1:** 允许运行时配置鲁棒性相关参数
理由: 根据实时监控数据动态优化系统, 提升灵活性
- 决策 2:** 支持对输入验证规则、威胁特征库的动态更新
理由: 确保输入验证和防护措施能持续应对新出现的漏洞和攻击模式

技术选择 (Choice of Technology) :

- 决策 1:** 采用抗干扰能力强的算法技术
理由: 从算法层面提升 AI 模型系统的鲁棒性
- 决策 2:** 进行故障注入测试, 主动模拟组件失效、网络延迟等异常场景
理由: 提前发现系统设计中的薄弱环节, 验证鲁棒性策略的有效性, 确保系统在真实异常环境下的稳定性